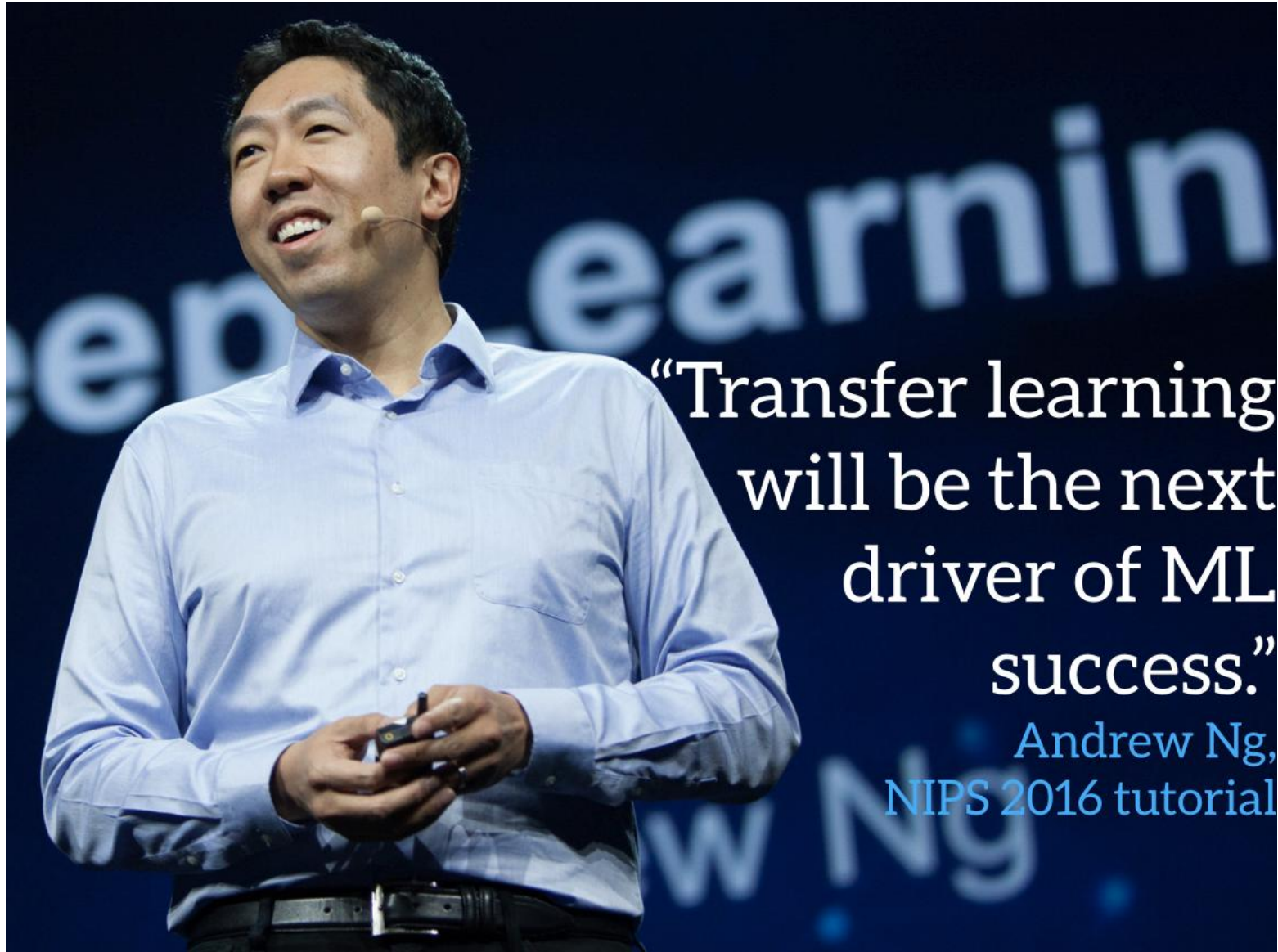


Transfer Learning



Transfer Learning



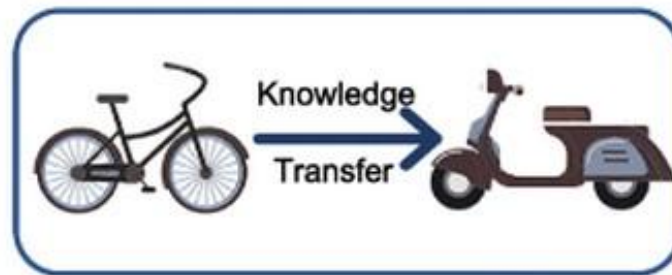
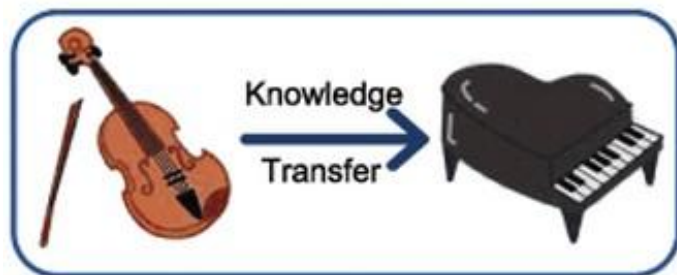
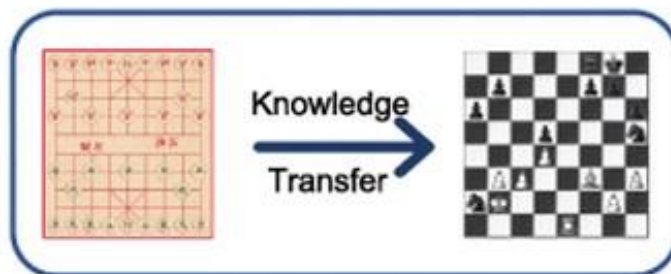
“Transfer learning
will be the next
driver of ML
success.”

Andrew Ng,
NIPS 2016 tutorial

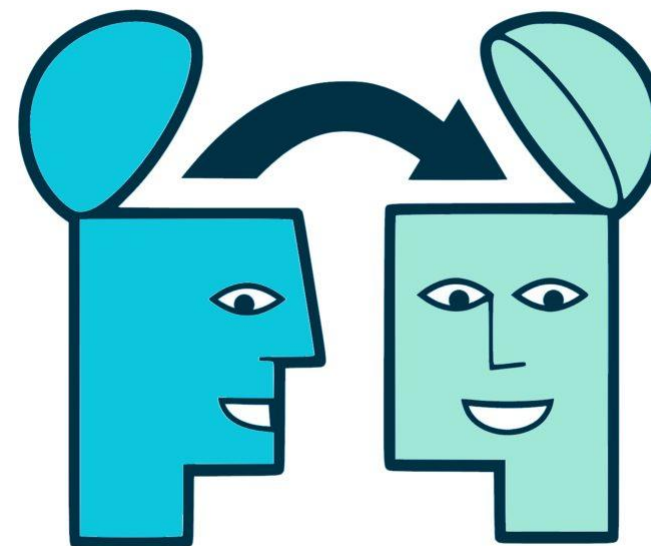
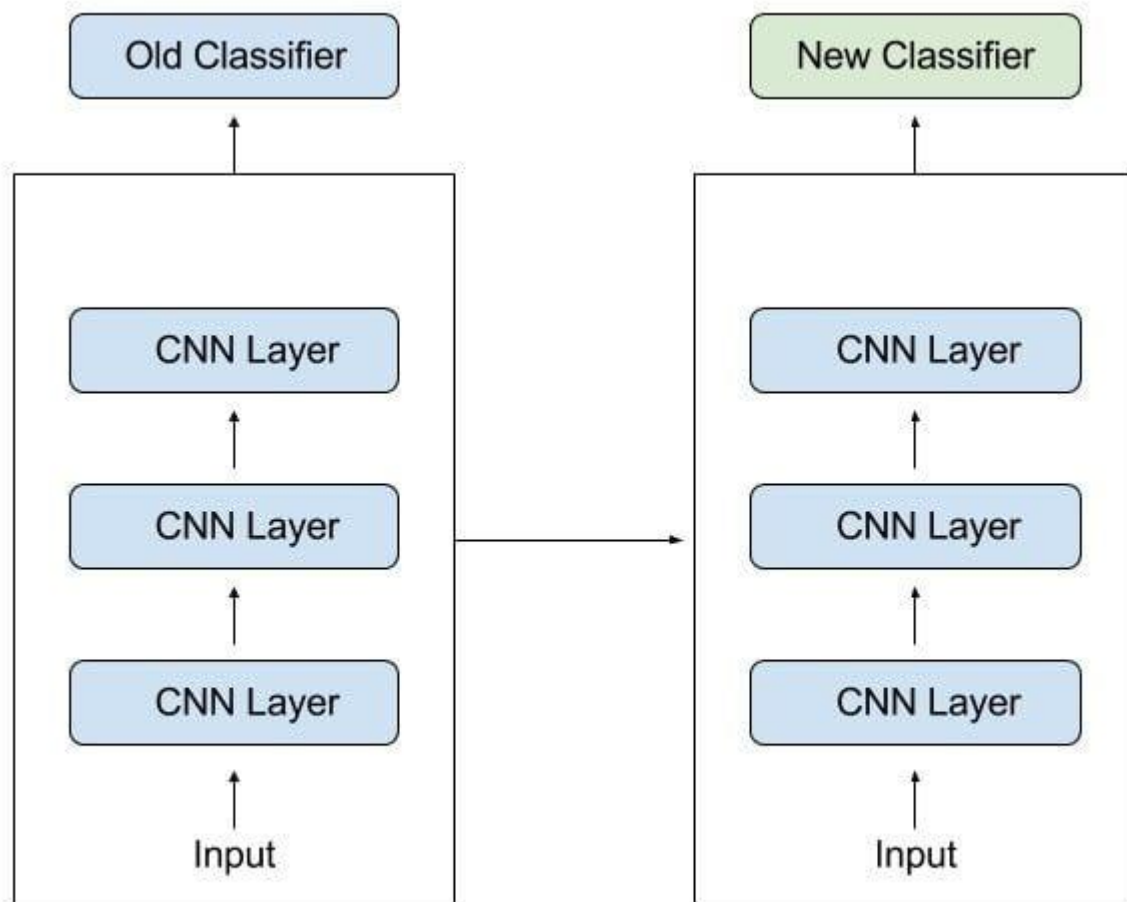
Transfer Learning



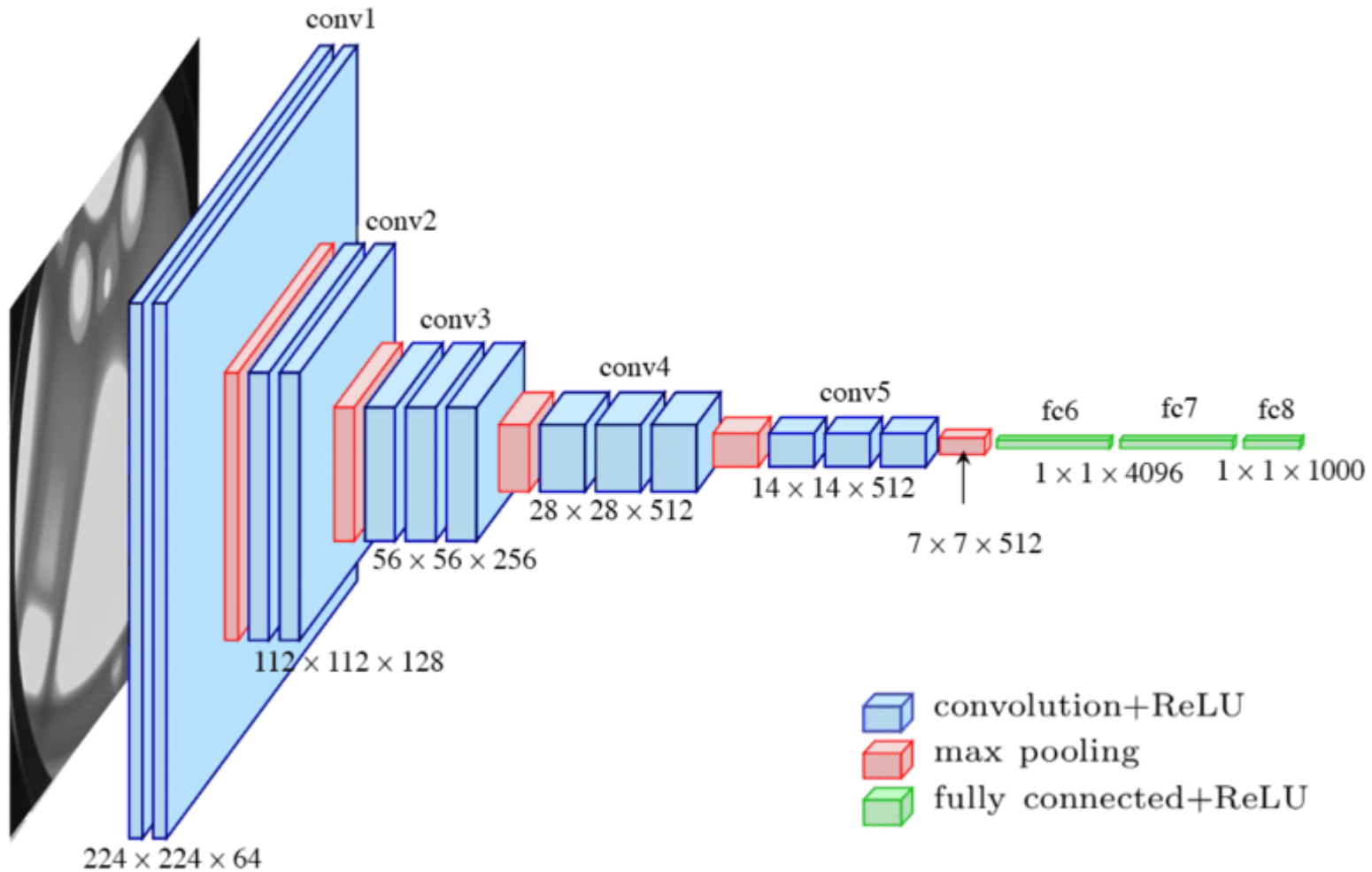
Transfer Learning



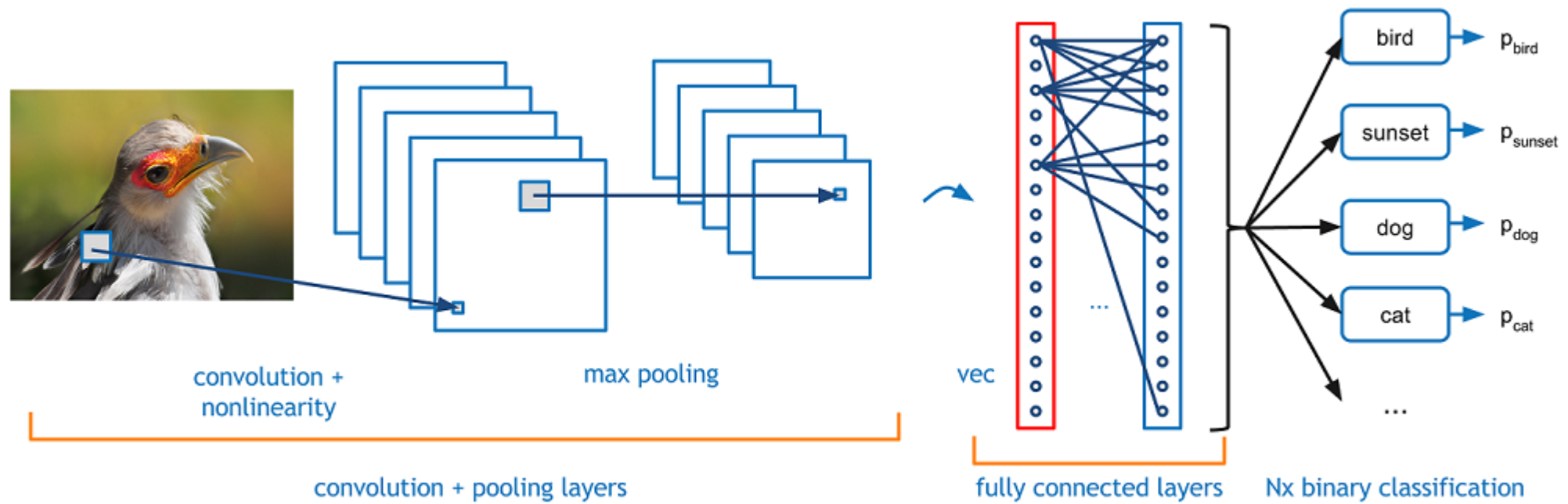
Transfer Learning



VGG Net

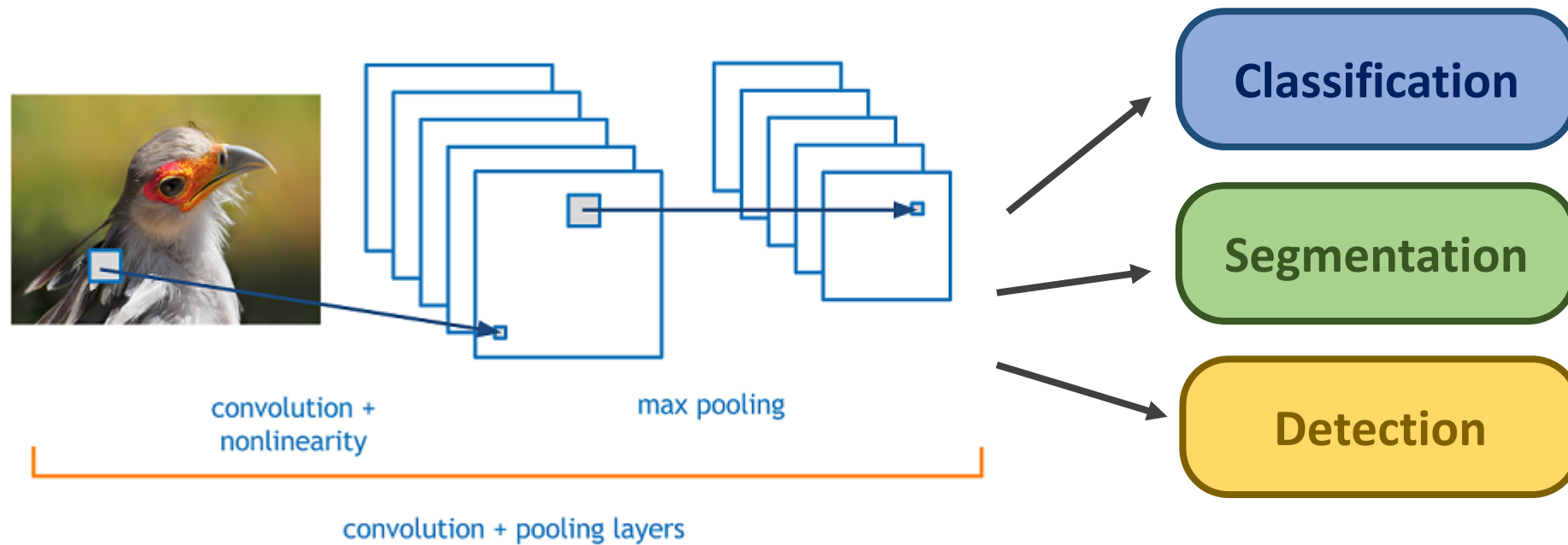


Transfer Learning



Transfer Learning

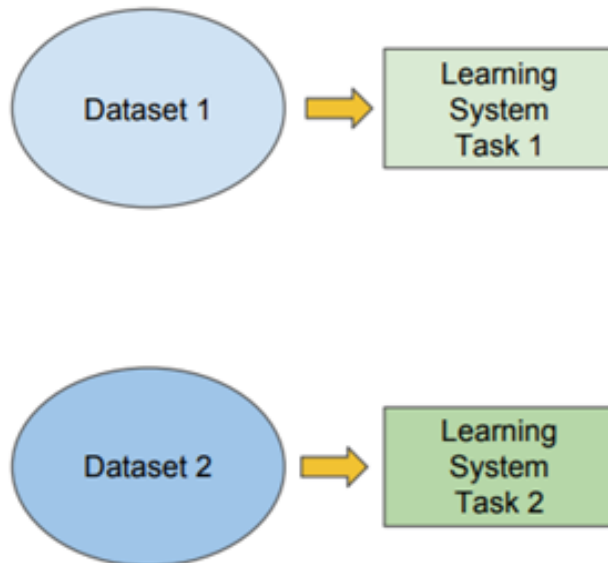
Backbone



Summary

Traditional ML

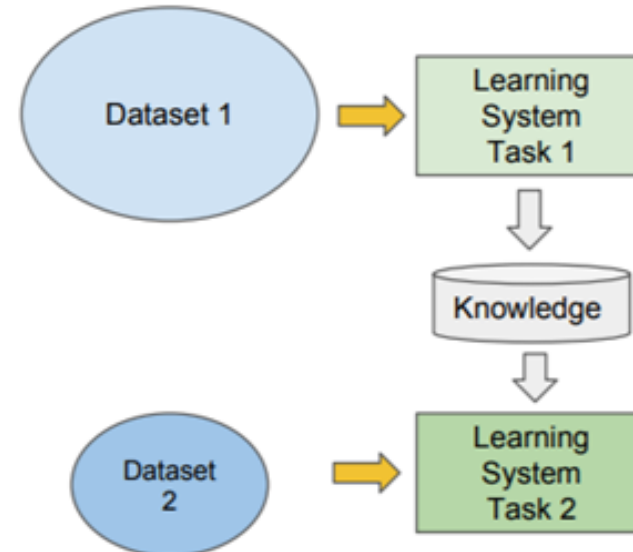
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

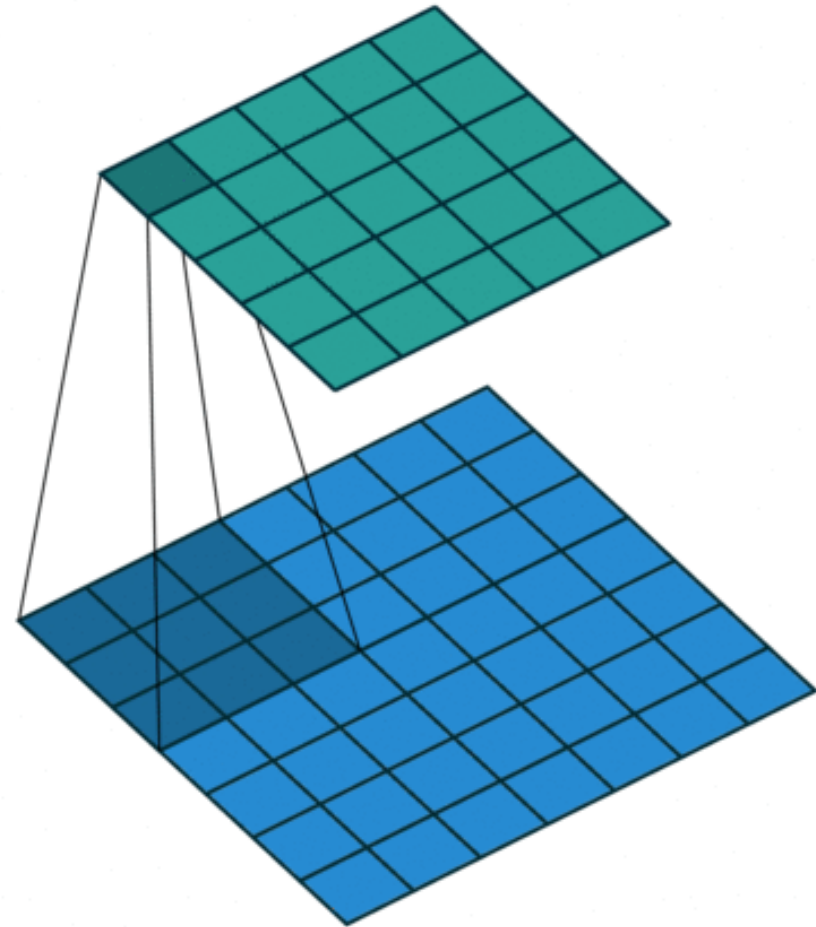
Transfer Learning

- Learning of a new task relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



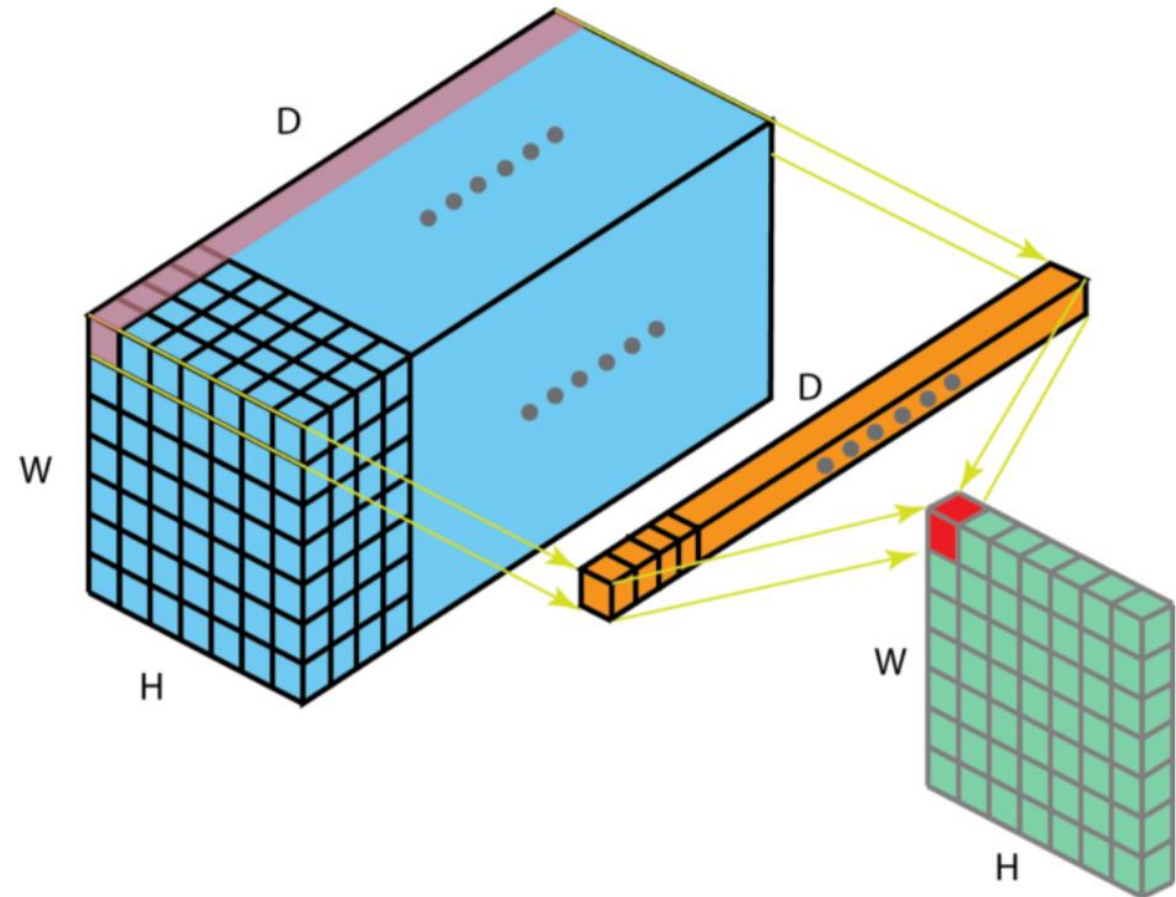
Types of Conv Layer

- **Standard Conv Layer**
 - Weight Sharing
 - Translation invariant



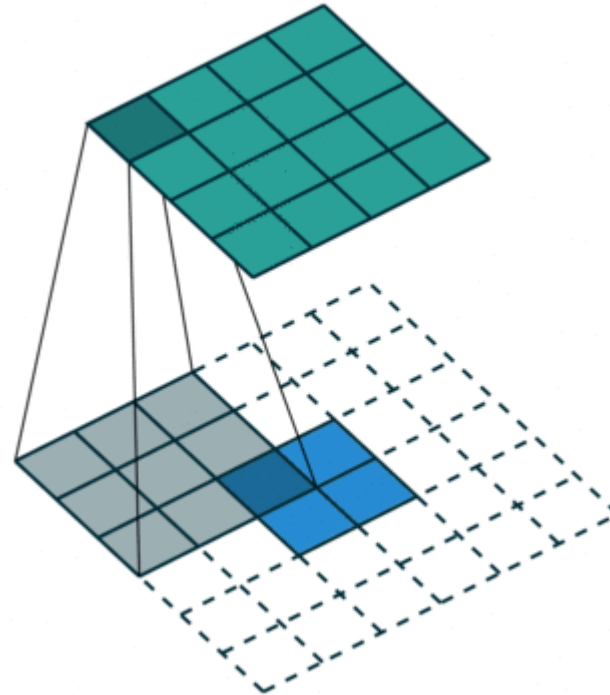
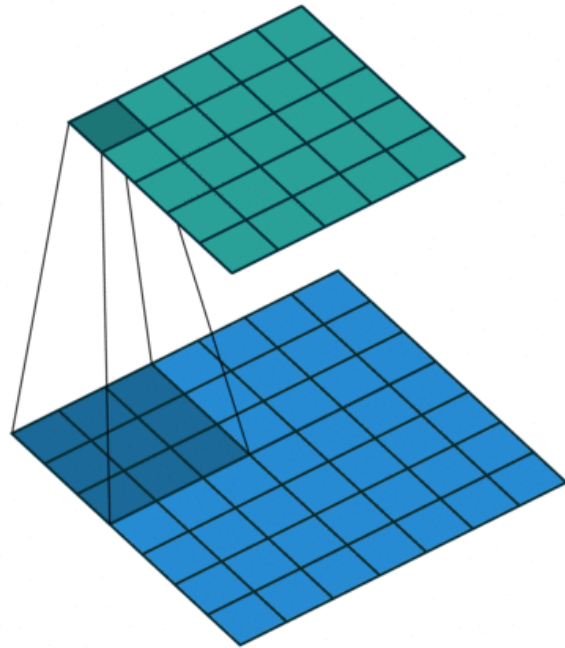
Types of Conv Layer

- **Point-wise Conv**



Types of Conv Layer

- **Transposed Conv**
 - Used for Upsampling
 - Other name: **Deconvolution, Fractionally strided Convolution**



Types of Conv Layer

some classic method for up sampling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

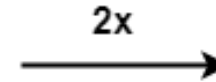
Input: 2 x 2

Output: 4 x 4

Bi-Linear Interpolation

10	20
30	40

2x2



2x

10	12	17	20
15	17	22	25
25	27	32	35
30	32	37	40

4x4

Types of Conv Layer

some classic method for up sampling

“Bed of Nails”

1	2
3	4

Input: 2 x 2



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



Rest of the network

Max unpooling

Max Unpooling

Use positions from pooling layer

1	2
3	4

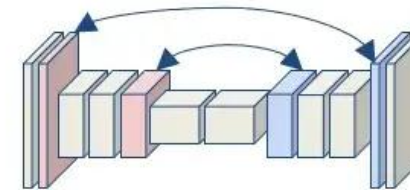
Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

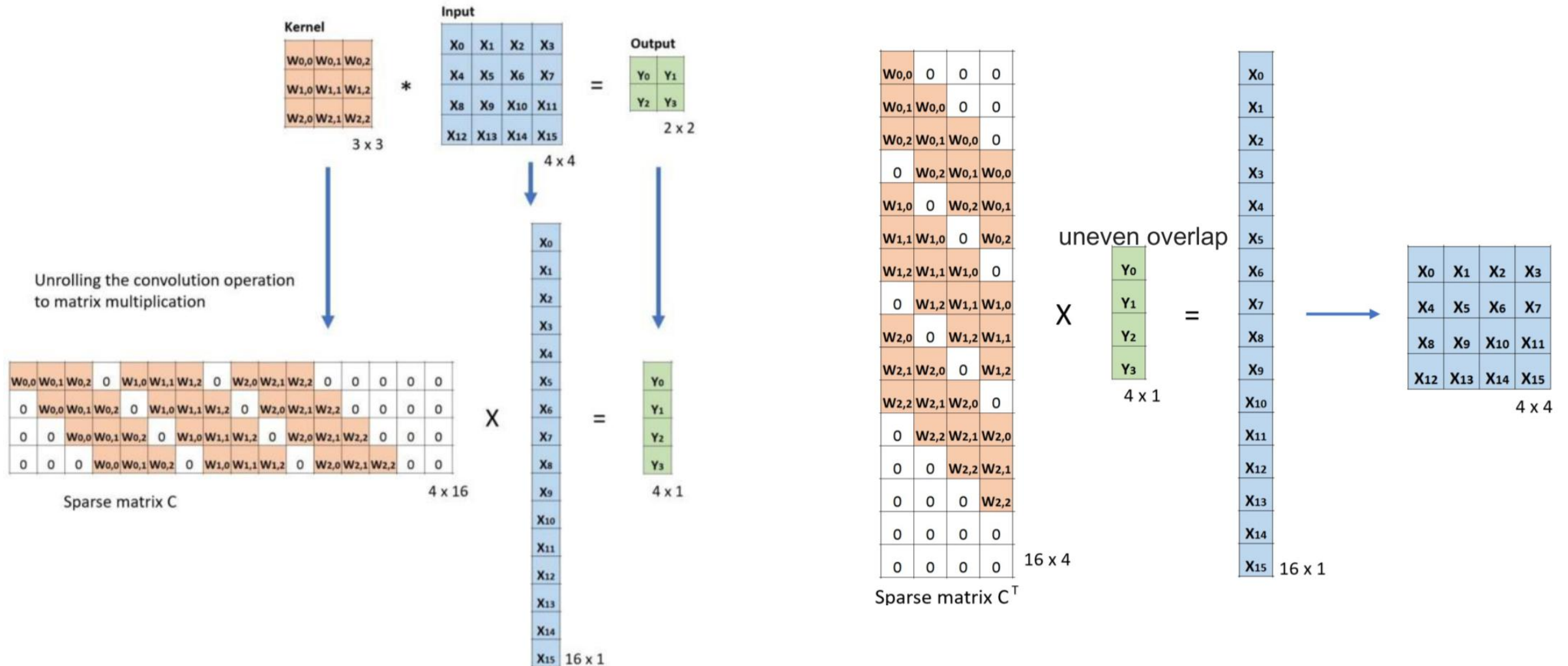
Output: 4 x 4

Corresponding pairs of
downsampling and
upsampling layers

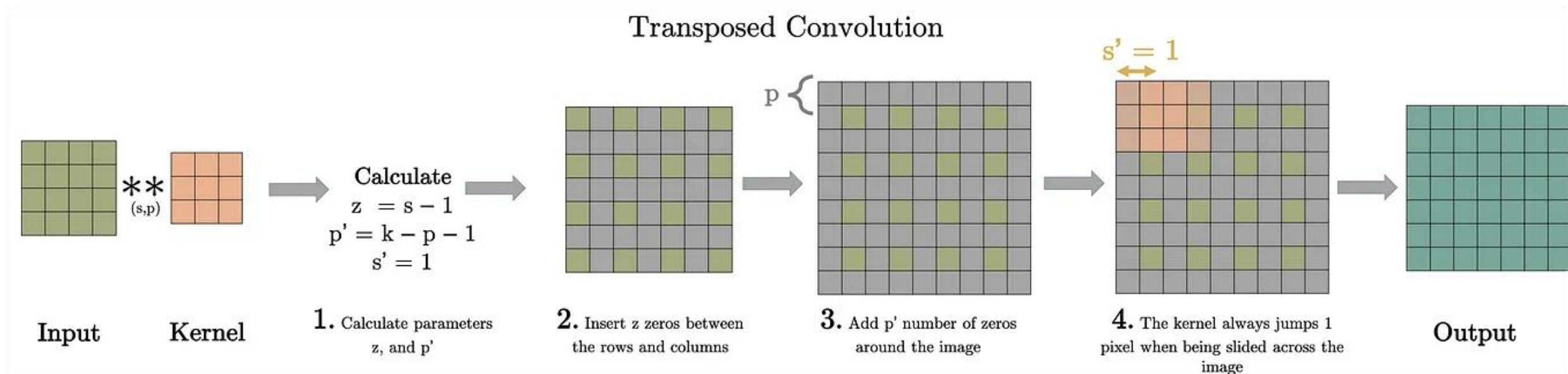
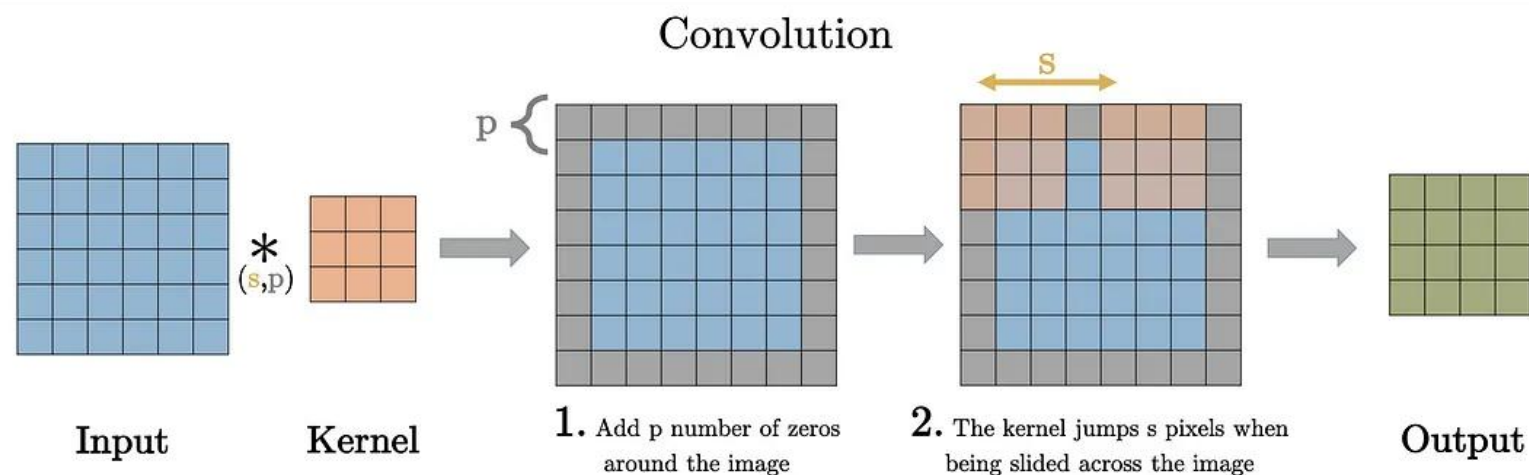


Transposed Convolution

<https://github.com/christianversloot/machine-learning-articles/blob/main/understanding-transposed-convolutions.md>



Transposed Conv implementation



Transposed Conv implementation



```
tf.keras.layers.Conv2DTranspose(filters_depth, filter_size, strides=(1, 1),  
padding='valid', output_padding=0)
```



```
torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size,  
stride=1, padding=0, output_padding=0, groups=1, bias=True, dilation=1,  
padding_mode='zeros', device=None, dtype=None)
```

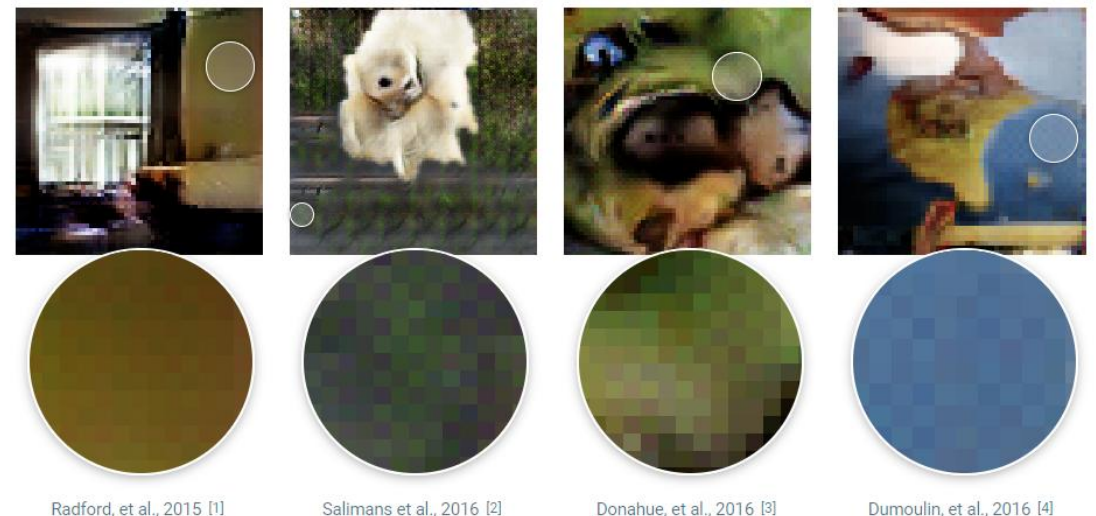
Checkerboard effects

Deconvolution and Checkerboard Artifacts

AUGUSTUS ODENA | VINCENT DUMOULIN | CHRIS OLAH | Oct. 17 | Citation:
Google Brain | Université de Montréal | Google Brain | 2016 | Odena, et al., 2016

When we look very closely at images generated by neural networks, we often see a strange checkerboard pattern of artifacts. It's more obvious in some cases than others, but a large fraction of recent models exhibit this behavior.

<https://distill.pub//draobrekcehc-vnosed/2016>

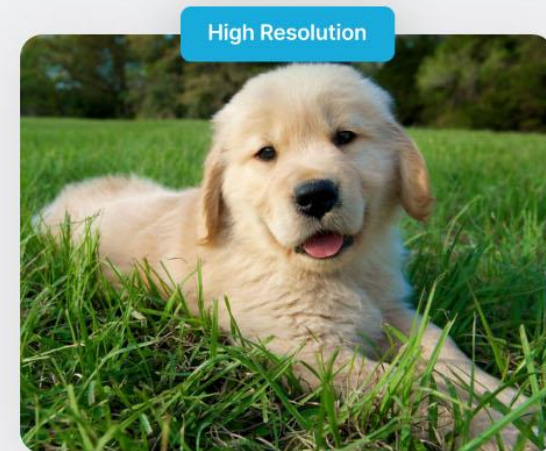
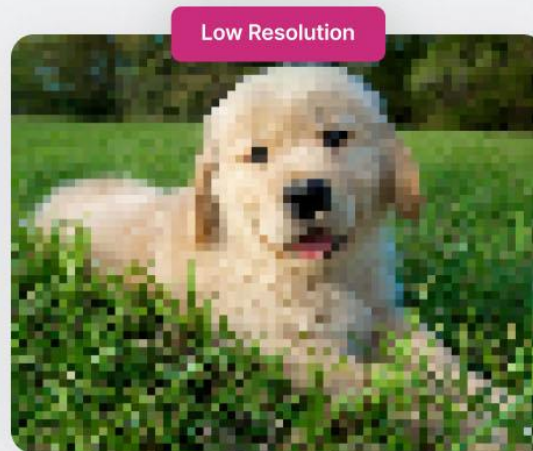


Applications

- Super- Resolution

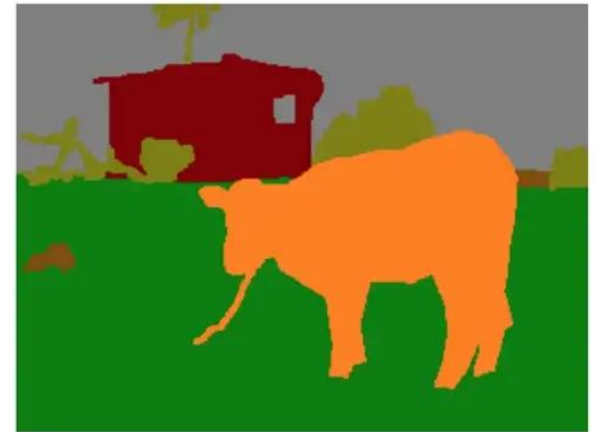


Charlie 777



Applications

- Semantic Segmentation



sky tree road grass water bldg mntn fg obj.

Types of Conv Layer

- **Dilated Conv (Atrous conv)**

- Dilated convolution with dilation rate L , insert $L-1$ zeros between kernel rows and columns



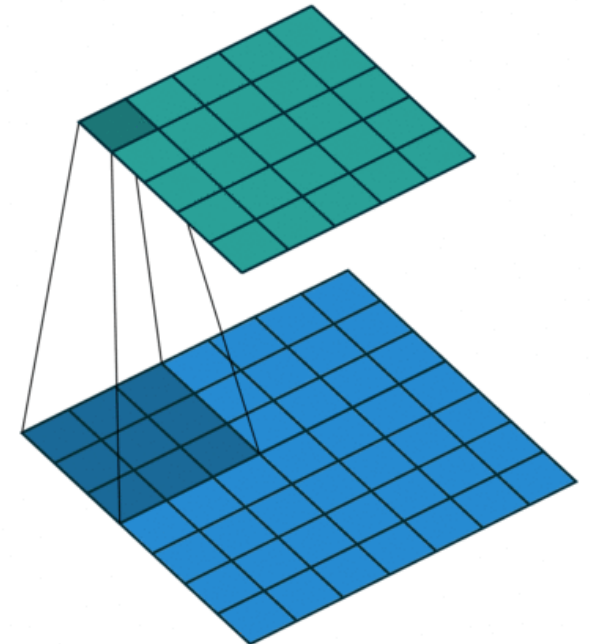
Conv2D layer

Conv2D class

[source]

```
tf.keras.layers.Conv2D(  
    filters,  
    kernel_size,  
    strides=(1, 1),  
    padding="valid",  
    data_format=None,  
    dilation_rate=(1, 1),  
    groups=1,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    **kwargs  
)
```

**Increase receptive
field without
increase kernel size**



Dilated Convolution

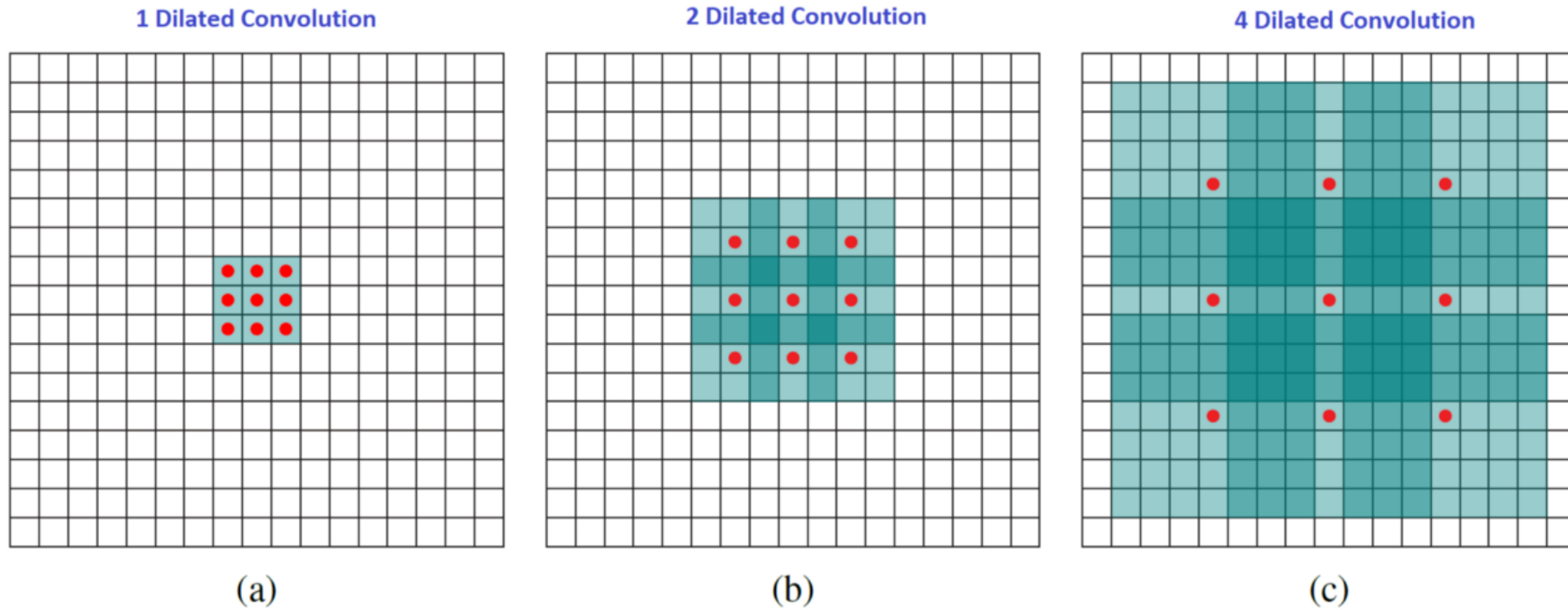


Figure 1: Systematic dilation supports exponential expansion of the receptive field without loss of resolution or coverage. (a) F_1 is produced from F_0 by a 1-dilated convolution; each element in F_1 has a receptive field of 3×3 . (b) F_2 is produced from F_1 by a 2-dilated convolution; each element in F_2 has a receptive field of 7×7 . (c) F_3 is produced from F_2 by a 4-dilated convolution; each element in F_3 has a receptive field of 15×15 . The number of parameters associated with each layer is identical. The receptive field grows exponentially while the number of parameters grows linearly.

Dilated Convolution

Other Advantages:

1. **Larger receptive field** (i.e. no loss of coverage)
2. **Computationally efficient** (as it provides a larger coverage on the same computation cost)
3. **Lesser Memory consumption** (as it skips the pooling step) implementation
4. **No loss of resolution of the output image** (as we dilate instead of performing pooling)
5. **Structure of this convolution helps in maintaining the order of the data.**

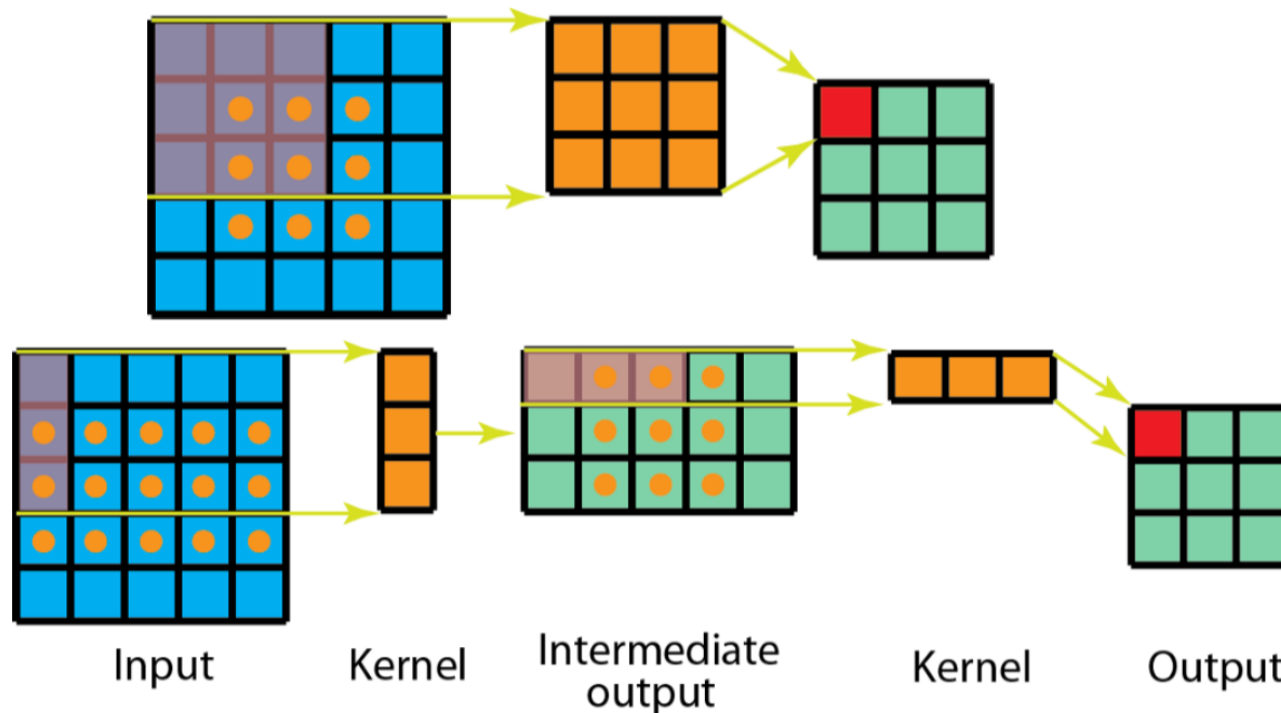
Types of Conv Layer

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

- **Separable Convolution**
 - Spatially Separable Convolution
 - Decrease number of computations

1) $3 \times 3 \times 1 \times 3 \times 3 \times 1 = 81$

2) $3 \times 5 \times 1 \times 3 \times 1 \times 1 + 3 \times 3 \times 1 \times 1 \times 3 \times 1 = 72$



Can every matrix be represented by two vectors/matrices?

Types of Conv Layer

- **Separable Convolution**
 - Depth wise Separable Convolutions
 - Its consists : Depth wise and $1*1$ convolution

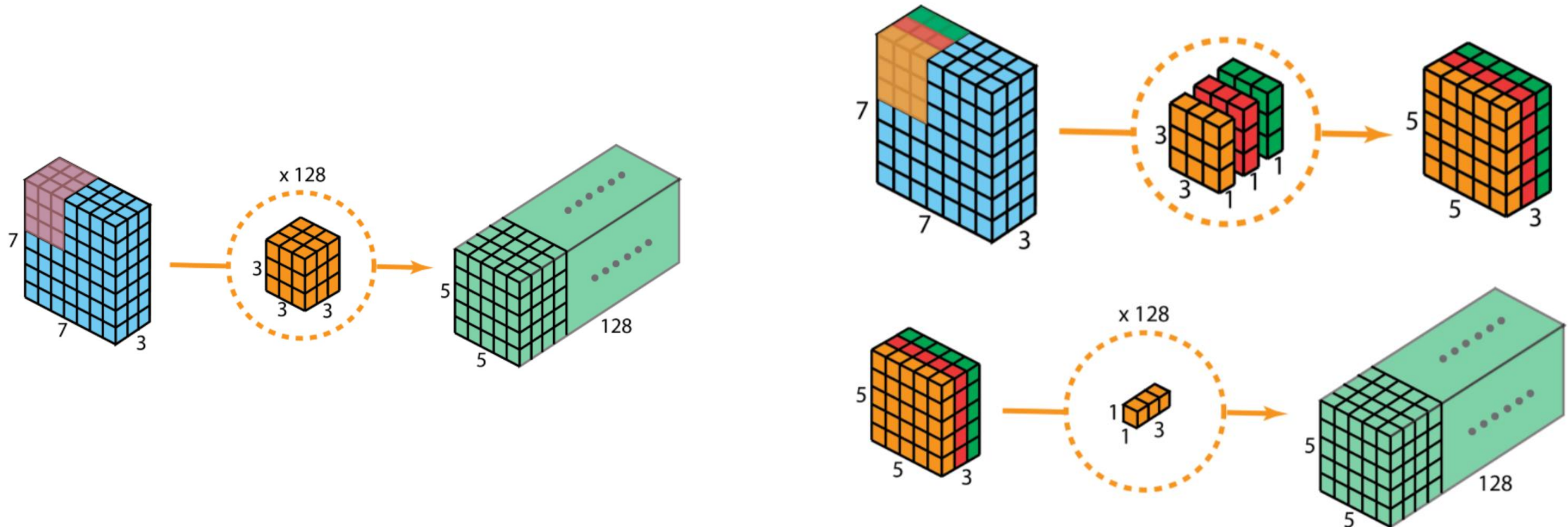
1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

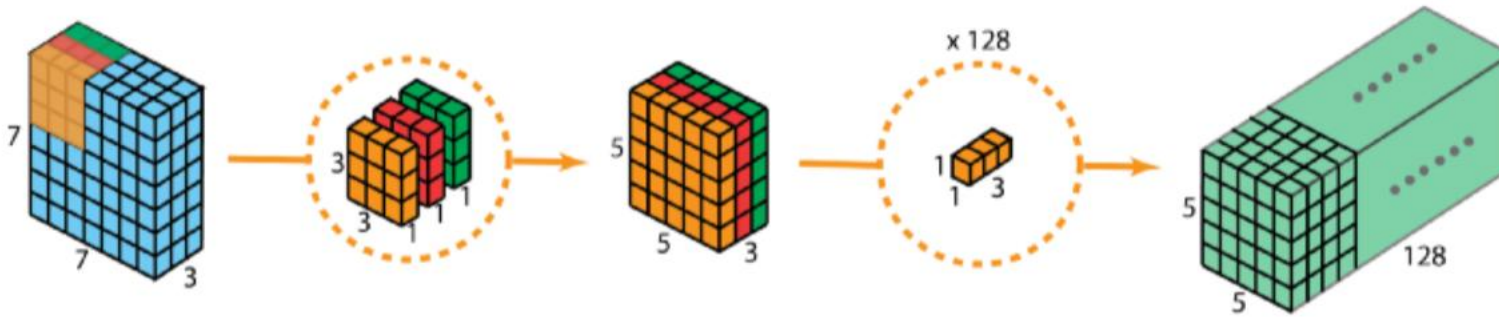
Convolved
Feature

Mul or Add?



Types of Conv Layer

- **Separable Convolution**
 - Depth wise Separable Convolutions
 - Its consists : Depth wise and 1×1 convolution



Summary

input tensor of size : $8 \times 8 \times 3$

Kernel size : $5 \times 5 \times 3$

output tensor is of size : $8 \times 8 \times 256$

Number of multiplications required ?

Standard Conv

$$(8 \times 8) \times (5 \times 5 \times 3) \times (256) = 1,228,800$$

Depth-wise Separable Convolutions

Filtering — Split into single channels, so $5 \times 5 \times 1$ filter is required in place of $5 \times 5 \times 3$, and since there are three channels, so the total number of $5 \times 5 \times 1$ filters required is 3, so,
 $(8 \times 8) \times (5 \times 5 \times 1) \times (3) = 3,800$

Combining — Total number of channels required is 256, so,
 $(8 \times 8) \times (1 \times 1 \times 3) \times (256) = 49,152$

$$\text{Total number of multiplications} = 3,800 + 49,152 = 53,952$$

$$1,228,800 / 53,952 = 23 \times \text{less multiplications required}$$



MobileNet



Figure 1. MobileNet models can be applied to various recognition tasks for efficient on device intelligence.

MobileNet v1

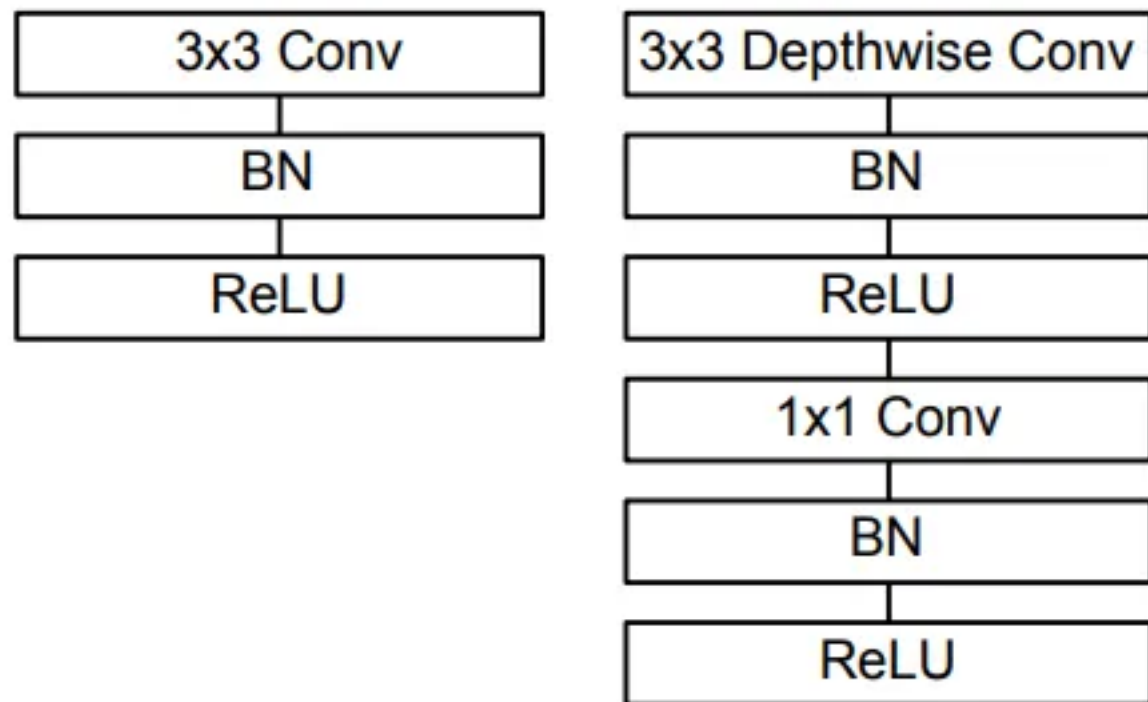


Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5 \times$	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

MobileNet v1

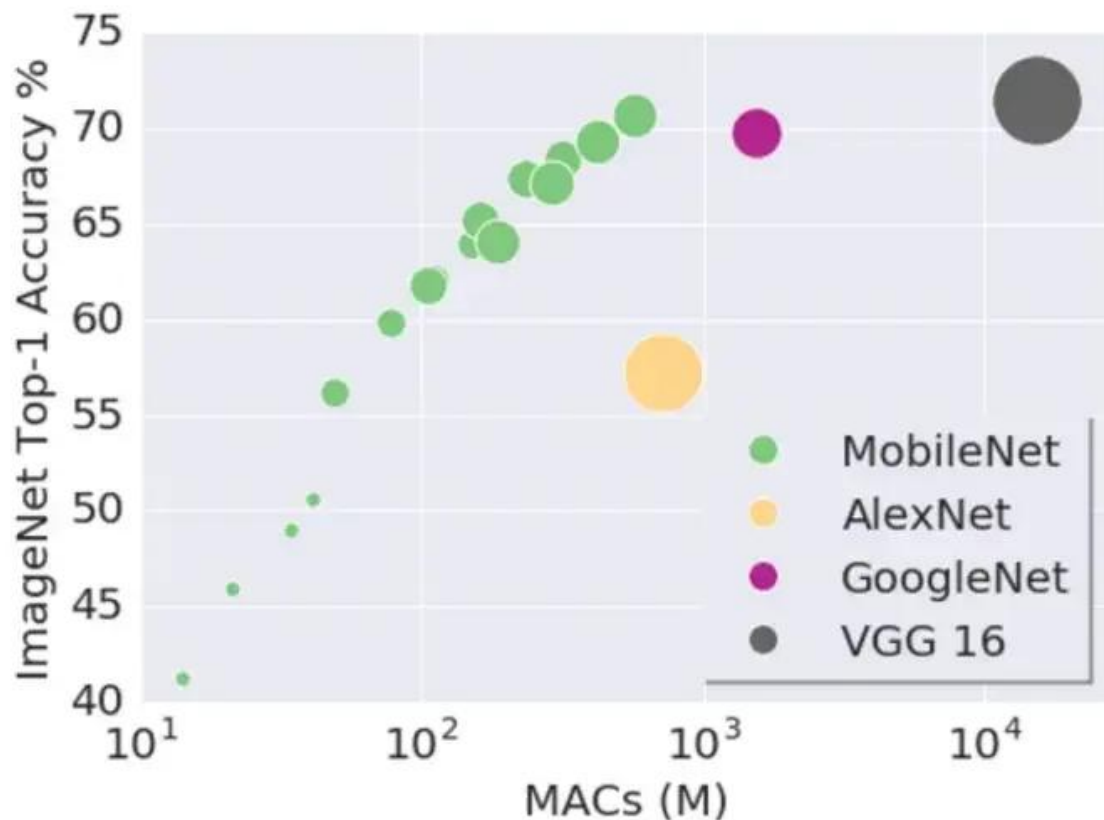


Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5 \times$	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

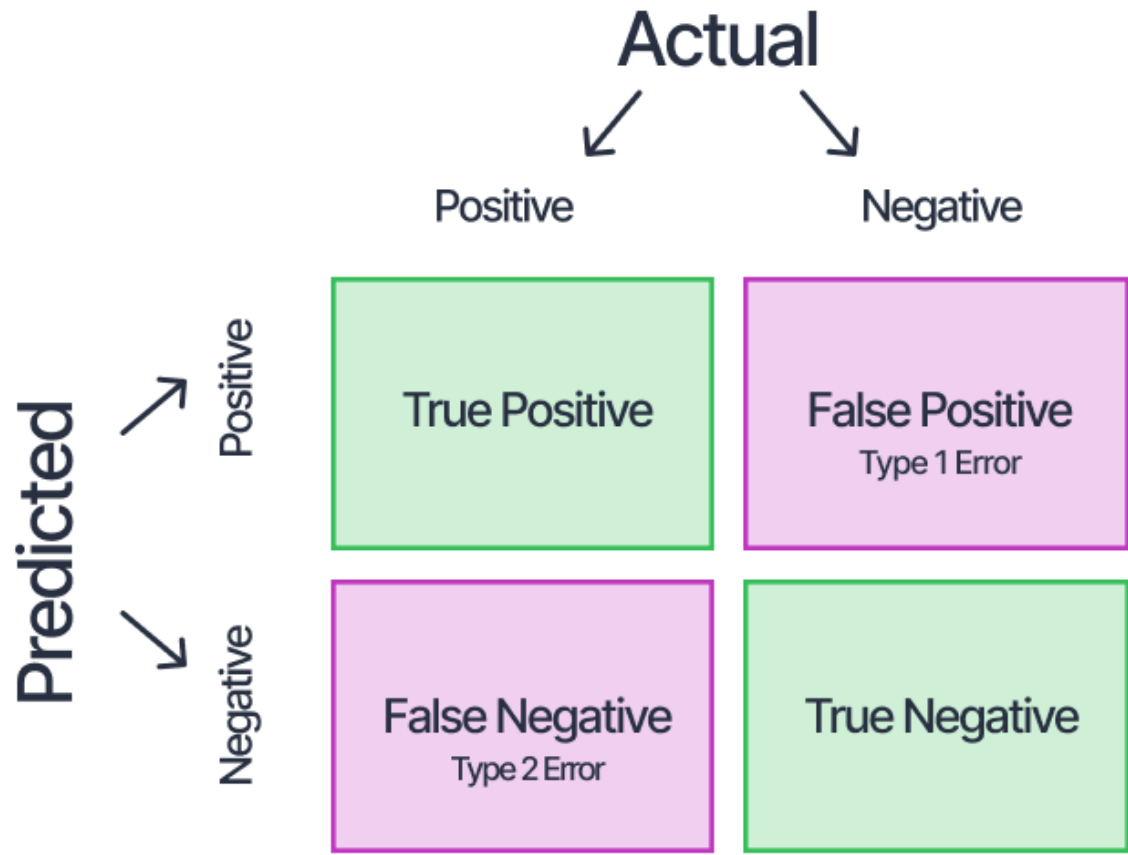
Exercise

- MobileNet v1 (paper)
- MobileNet v2
- NasNet
- MobileNet v3
- Efficient Net V1
- Efficient Net V2

Evaluation Metrics



Exercise

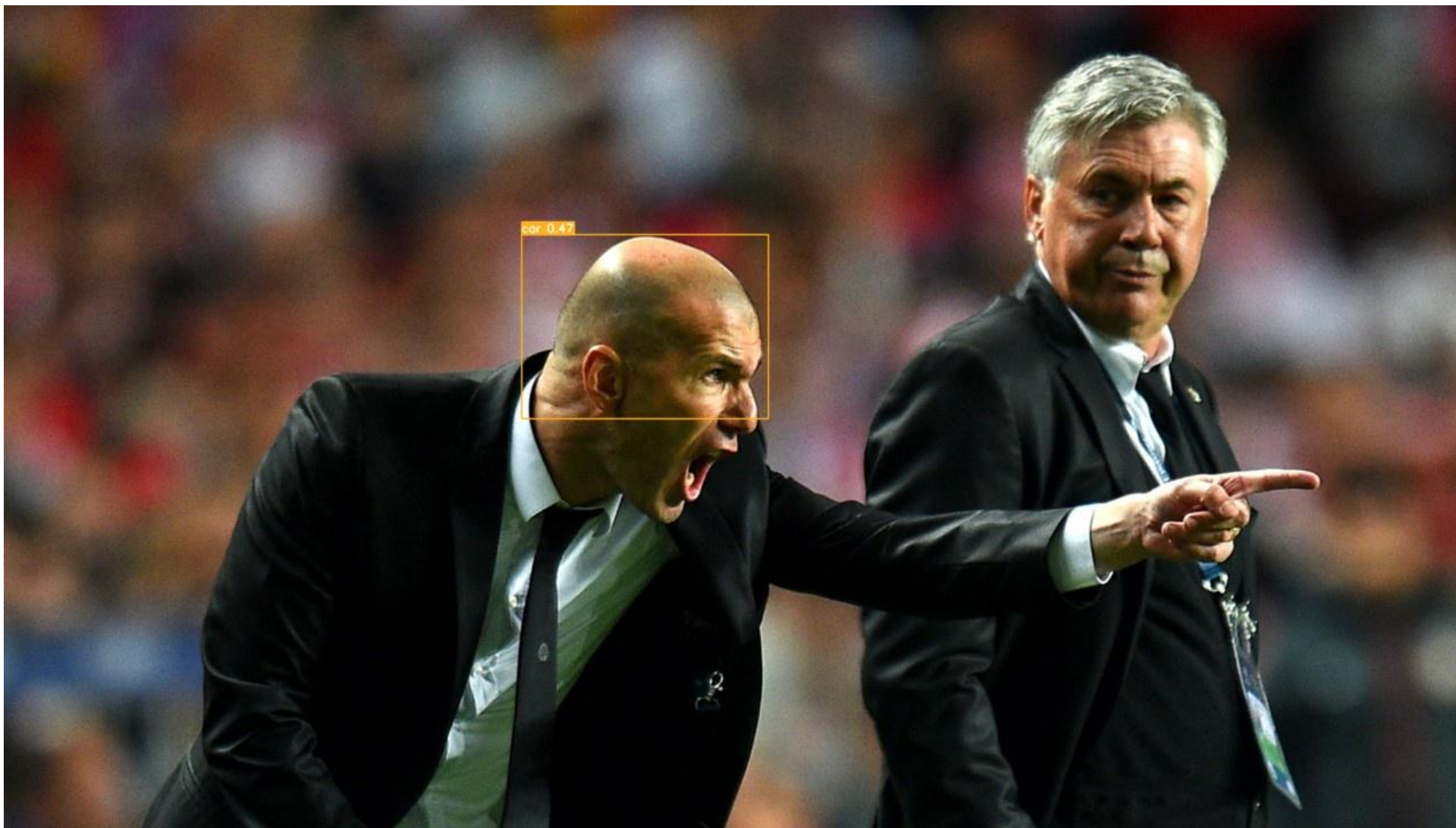


$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

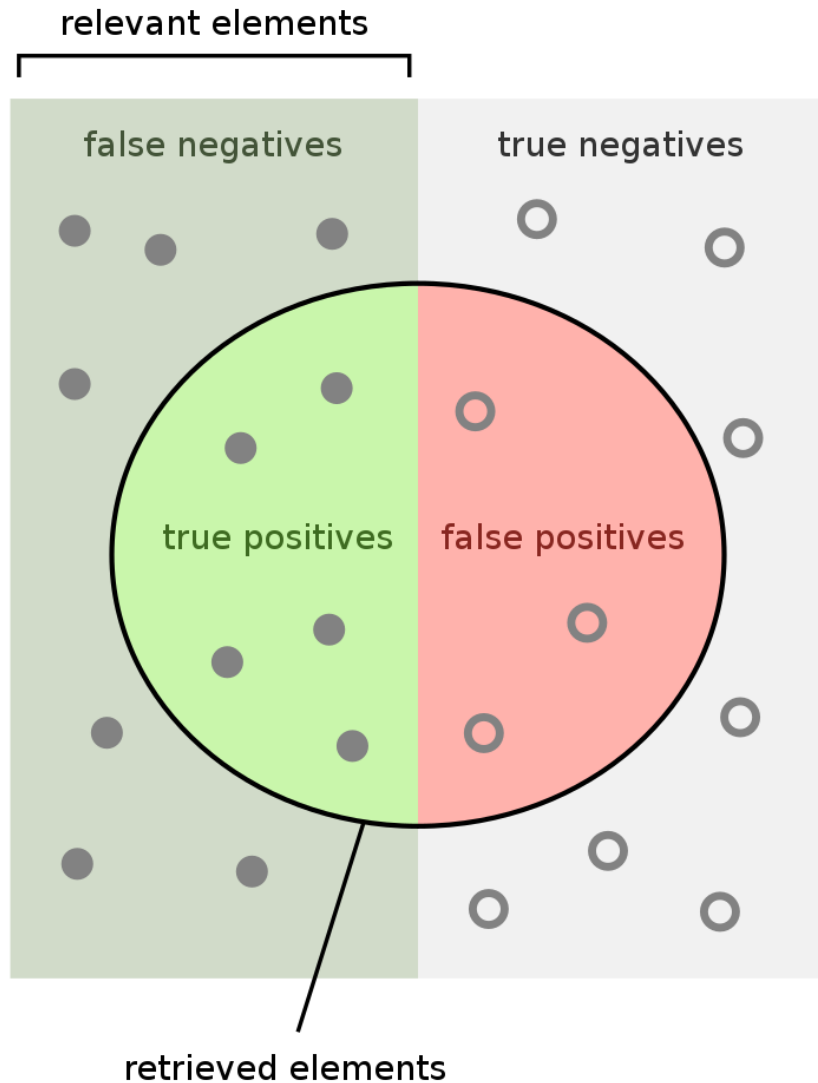
Exercise



Exercise



Summary



How many retrieved items are relevant?

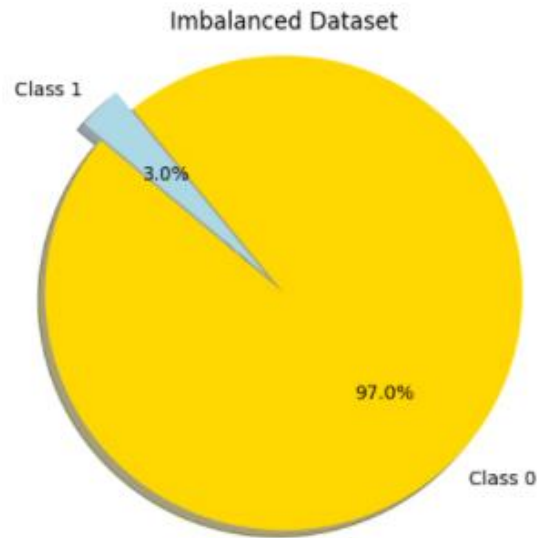
$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are retrieved?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$F1 \text{ score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Evaluation Metrics



Class 1 : Positive

Class 0 : Negative

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

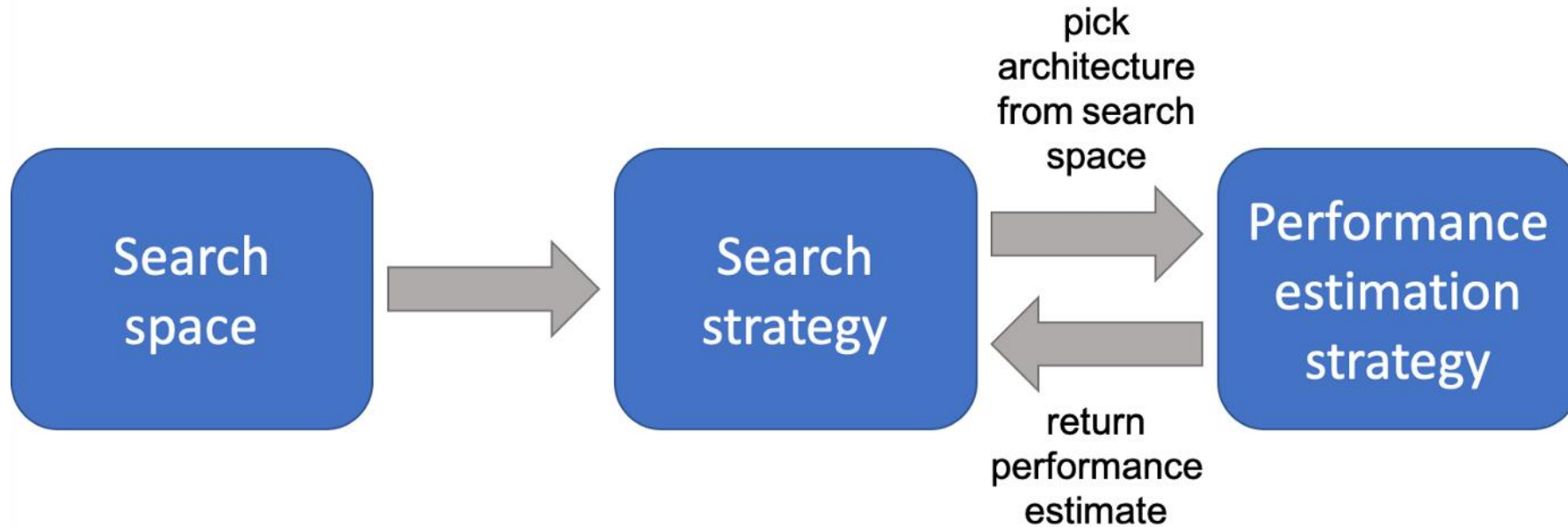
Neural Architecture Search (NAS)

A Neural Network created by an algorithm



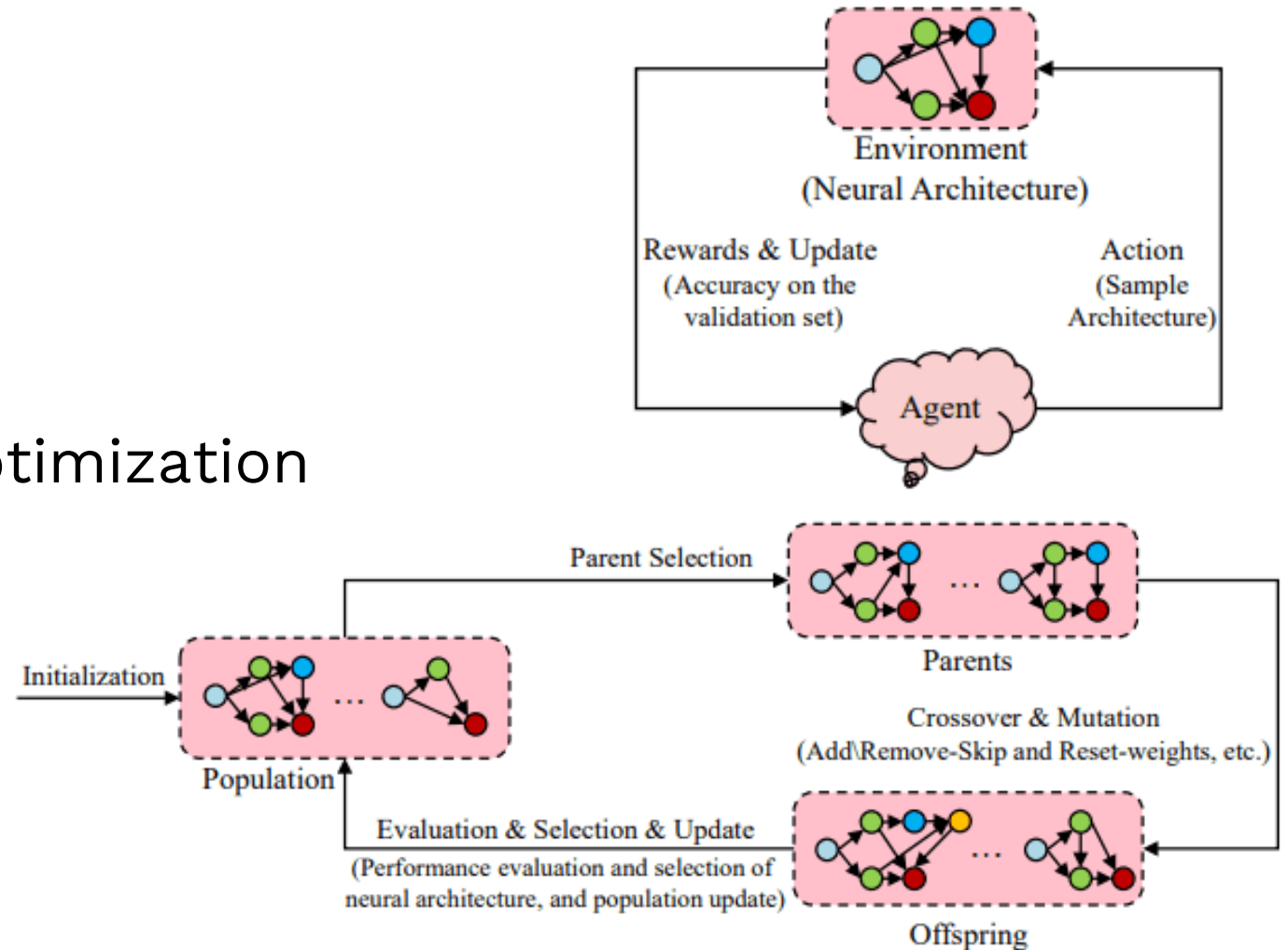
Summary

<https://theaisummer.com/neural-architecture-search/>



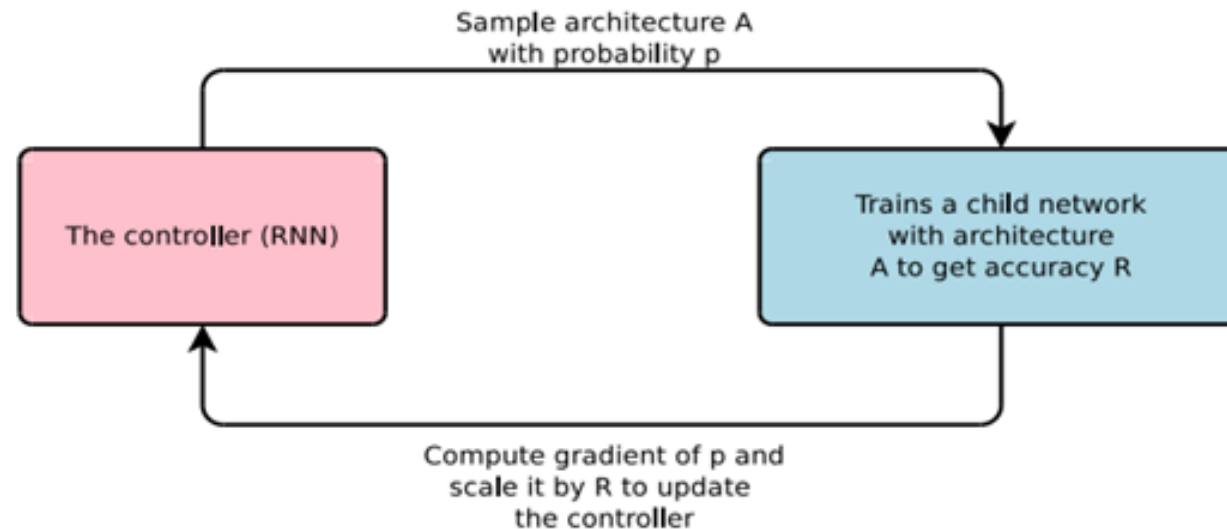
Summary

1. Random search
2. Reinforcement learning
3. Evolutionary algorithms
4. Sequential model-based optimization
5. Gradient optimization



NAS Net paper

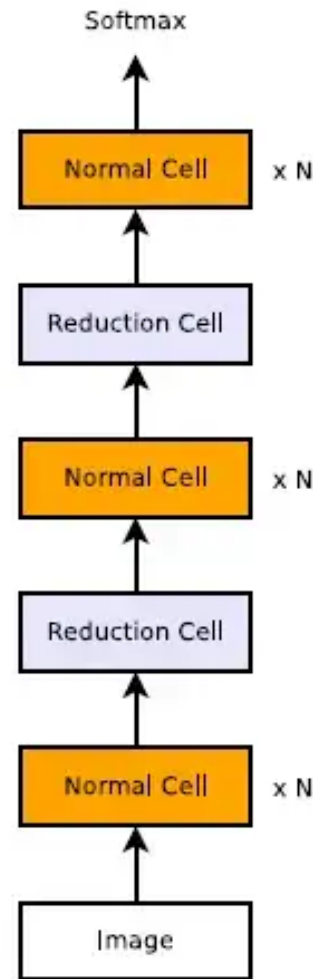
- Search for an architectural building block on a small dataset
- Transfer the block to a larger dataset



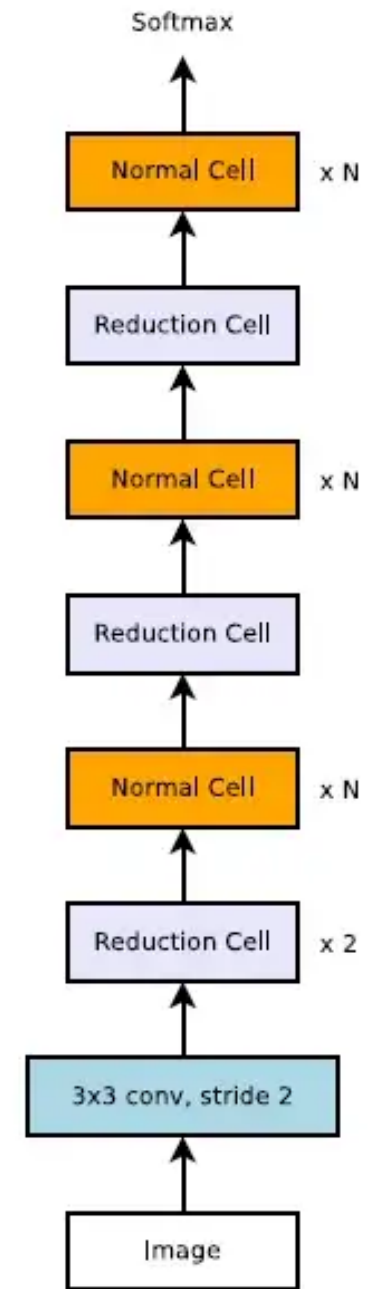
Summary

Normal Cell: Convolutional cells that return a feature map of the same dimension.

Reduction Cell: Convolutional cells that return a feature map where the feature map height and width is reduced by a factor of two.

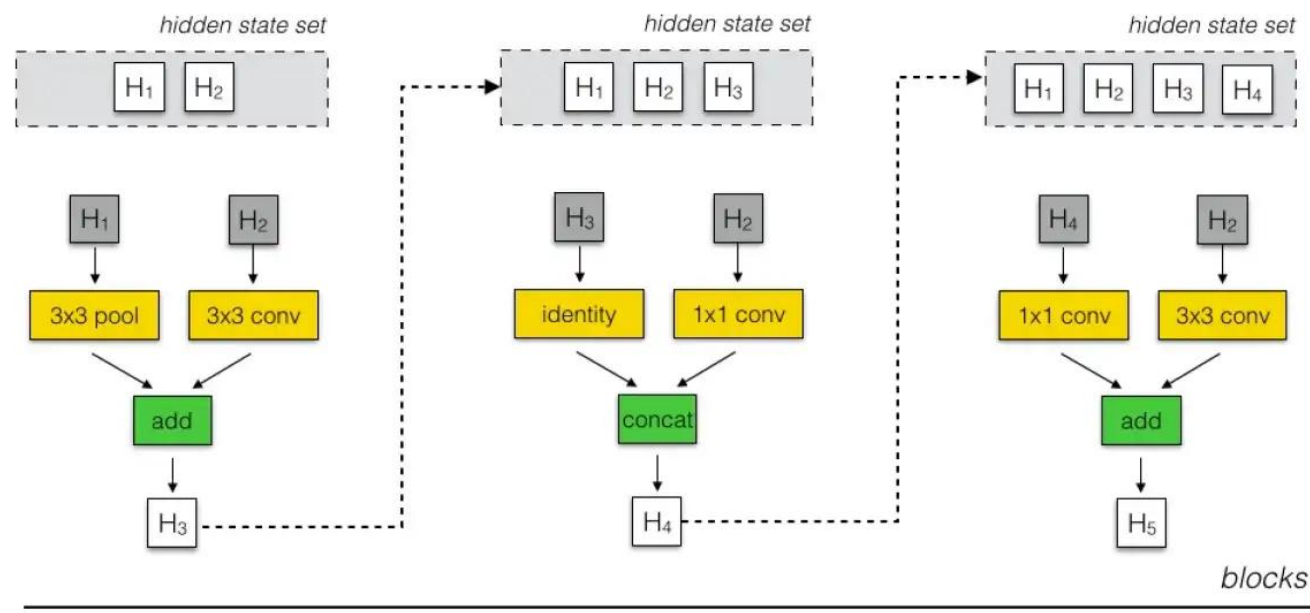
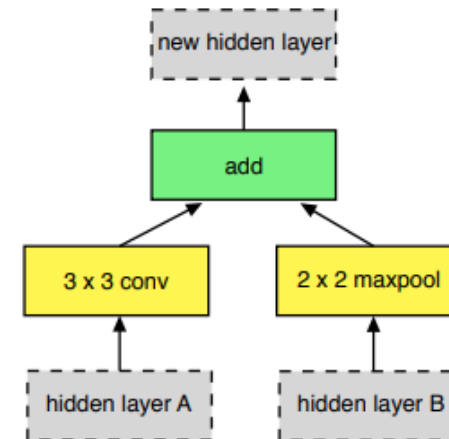
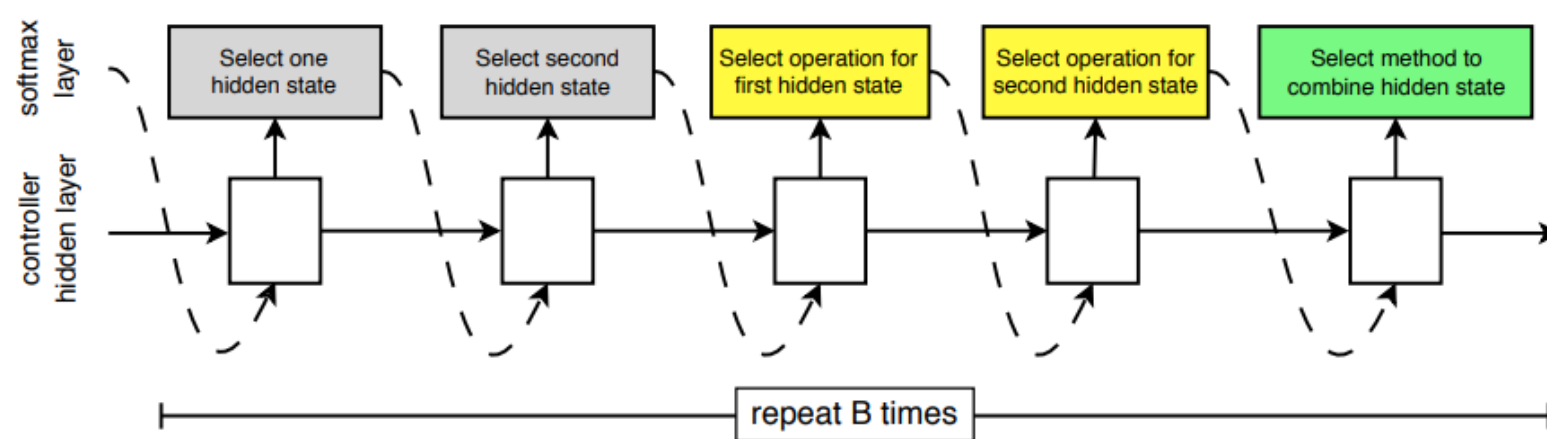


CIFAR10
Architecture



ImageNet
Architecture

Summary



Summary

Step 1: Select a hidden state from h_i , h_{i-1} or from the set of hidden states created in previous blocks.

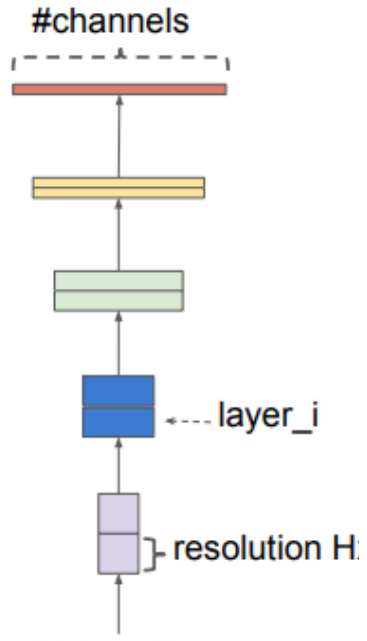
Step 2: Select a second hidden state from the same options as in Step 1.

Step 3: Select an operation to apply to the hidden state selected in Step 1.

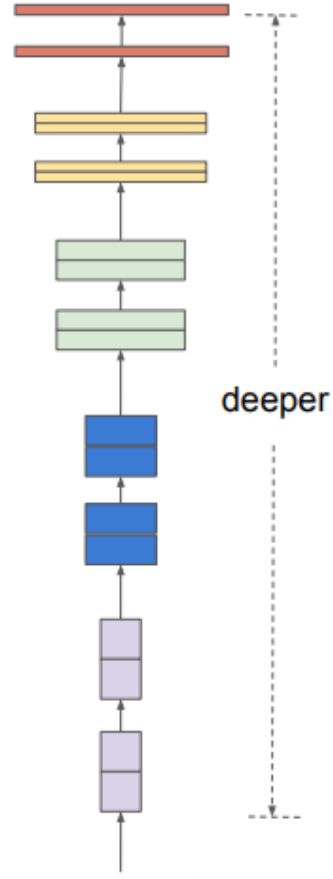
Step 4: Select an operation to apply to the hidden state selected in Step 2.

Step 5: Select a method to combine the outputs of Step 3 and 4 to create a new hidden state.

Efficient Net V1

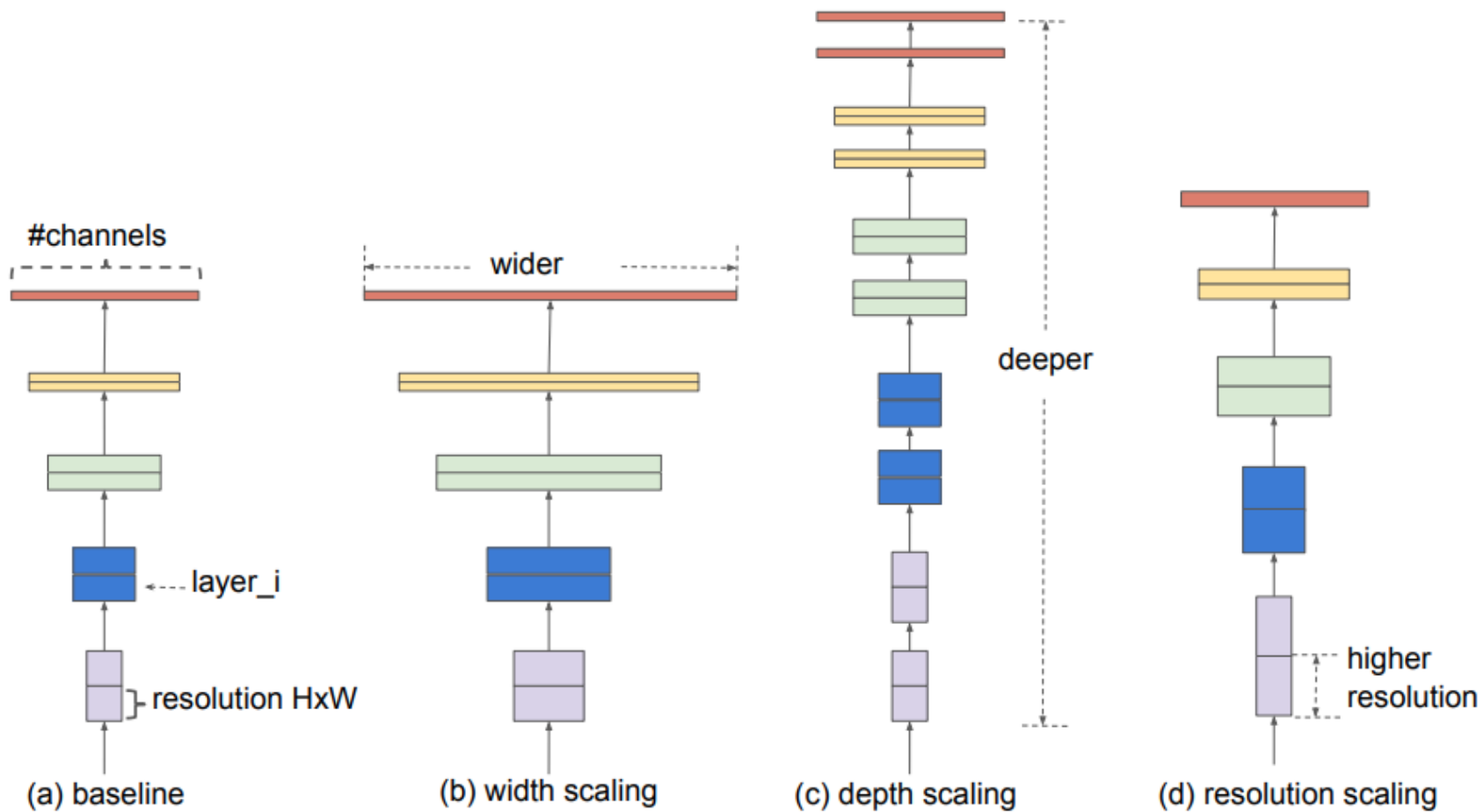


(a) baseline

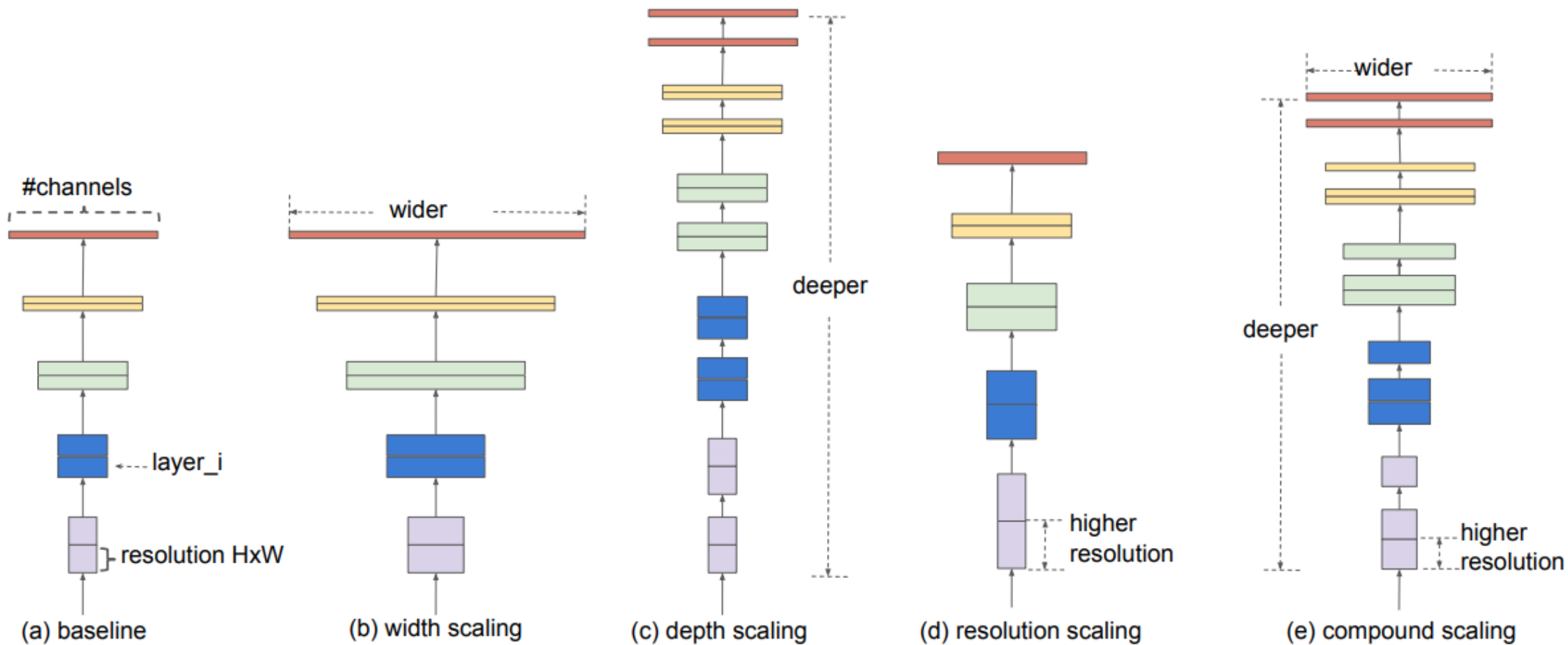


(c) depth scaling

Summary



Summary



Summary

EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

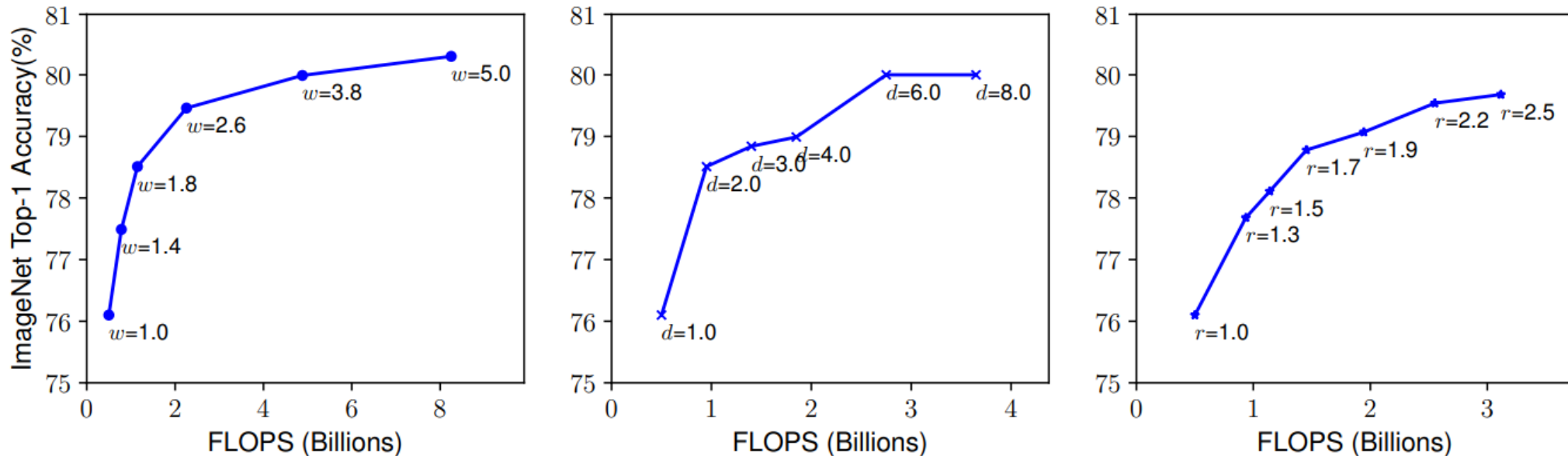


Figure 3. Scaling Up a Baseline Model with Different Network Width (w), Depth (d), and Resolution (r) Coefficients. Bigger networks with larger width, depth, or resolution tend to achieve higher accuracy, but the accuracy gain quickly saturate after reaching 80%, demonstrating the limitation of single dimension scaling. Baseline network is described in Table 1.

Summary

depth: $d = \alpha^\phi$

width: $w = \beta^\phi$

resolution: $r = \gamma^\phi$

s.t. $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$

$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$

Doubling network depth will double FLOPS, but doubling network width or resolution will increase FLOPS by four times

for any new ϕ , the total FLOPS will approximately increase by 2

- **STEP 1:** we first fix $\phi = 1$, assuming twice more resources available, and do a small grid search of α, β, γ based on Equation 2 and 3. In particular, we find the best values for EfficientNet-B0 are $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$, under constraint of $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$.
- **STEP 2:** we then fix α, β, γ as constants and scale up baseline network with different ϕ using Equation 3, to obtain EfficientNet-B1 to B7 (Details in Table 2).